First-Generation Neural Track Extrapolation for LHCb Allen HLT1: A 17-Architecture MLP Sweep on 50M Runge-Kutta-Generated Tracks

G. Scriven
Maastricht University, UHasselt, Nikhef

April 2026

Abstract

We report the results of the `gen_1` campaign to replace the analytic fourth-order Runge-Kutta (RK4) extrapolator in the LHCb *Allen* HLT1 reconstruction chain with a purely data-driven multi-layer perceptron (MLP). A dataset of 50e6 track segments, sampled uniformly in the LHCb VELO fiducial region and integrated through the measured dipole field, is used to train 17 MLP architectures covering the range 4.9k–1.06M parameters under a common SiLU/AdamW/cosine protocol. The best mean 3D position error reached is 1.02 mm (architecture `mlp_3x64`, 9028 parameters), with median below 1 mm; slope errors plateau at $\sim 4\times10^{-3}$ rad. We identify three blocking issues that motivate the follow-on `gen_2` work: (i) all sub-64 kB (constant-memory deployable) models are strictly above the Pareto front defined by the larger networks; (ii) a fixed-sample-budget confound — every run uses only 5e6 of the 50e6 available tracks — means the reported scaling is under-trained; (iii) slope errors remain too loose for direct ingestion by the downstream Kalman filter. A companion report, `gen2_old_data_results.tex`, introduces physics-informed baselines that attempt to close these gaps.

Executive summary

- **Dataset.** 50e6 track segments, RK4-integrated through a trilinearly interpolated $81 \times 81 \times 146$ dipole map; peak $|B_y| \approx 1.03\ T$ at $z \approx 5007\ mm$.
- **Model zoo.** 17 feedforward MLPs spanning 4868–1.06e6 parameters (hidden widths 64–1024, depths 2–4) trained with a common protocol: SiLU activation, AdamW (lr $= 10^{-3}$, wd $= 10^{-4}$), cosine schedule with 5-epoch warm-up, gradient clipping 1.0, batch size 4096, early stopping patience 30, `max_samples`$= 5e6$ across the board.
- **Best accuracy.** `mlp_3x64` (9k parameters): mean 3D position error 1.02 mm (median 0.73 mm; 95th percentile 2.97 mm). Best 95th percentile overall is `mlp_2x64` (4868 params) at 2.63 mm. Slope errors for all competitive models sit in the range $(2\text{–}8)\times 10^{-3}$ rad.
- **Pareto finding.** The 64 kB CUDA constant-memory budget (16384 float32 parameters) is *feasible*: three architectures fall under that threshold (`mlp_2x64`, `mlp_3x64`, `mlp_128_64`, `mlp_256_128` — the last borderline, see Table 2). They are, however, dominated by mid-size non-deployable networks on 95/99-percentile metrics.
- **Transition points motivating `gen_2`.** (a) Fixed 5e6-sample budget confounds capacity scaling; (b) deployment constraint (64 kB) collides with the accuracy-capacity frontier; (c) no physics constraints — the network learns a flow map by pure regression, with no enforcement of the Lorentz ODE.

1 Introduction

The LHCb experiment [1] at the CERN LHC triggers on beauty and charm decays in real time, reconstructing charged particle tracks at the 30 MHz bunch crossing rate of Run 3. Since 2022 the first-level software trigger (HLT1) has run entirely on GPUs via the *Allen* framework [2]; within this chain, track propagation through the LHCb dipole magnet is currently performed by an analytic Runge-Kutta (RK4) extrapolator applied to the Kalman-filter state vector [3]. The extrapolator is numerically exact relative to the underlying field map but is expensive: the RK4 inner loop requires many field lookups per track and dominates the cost of the track-fit stage on GPU.

Two parallel strategies have been pursued within the group to replace or accelerate this kernel. The first, ongoing in `experiments/field_maps/`, trains a compact neural network to approximate the *field map* $\mathbf{B}(\mathbf{r})$ itself, and feeds the network output into the existing RK4 integrator. The second strategy, which is the subject of this report, trains a neural network to approximate the full flow map $(\mathbf{y}_0, \Delta z) \mapsto \mathbf{y}(z_0 + \Delta z)$ end-to-end, replacing the RK4 integrator entirely with a single forward pass.

Both strategies are constrained by the same deployment budget: the trigger kernel must fit within the 64 kB of CUDA constant memory if the network weights are to be broadcast efficiently to all threads in a warp, which caps the parameter count at 16384 float32 values.

This report focuses on the end-to-end MLP strategy, specifically the `gen_1` V1 campaign: a 17-architecture sweep run under a uniform training protocol on a 50e6-track dataset. The goals are to (i) establish the accuracy baseline achievable by a direct regression model, (ii) map the Pareto frontier between parameter count and residual error, and (iii) identify the limitations that motivate the subsequent physics-informed extensions in `gen_2`.

2 Physics setup

2.1 z-parameterised equations of motion

A charged particle in a static magnetic field $\mathbf{B}(\mathbf{r})$ satisfies the Lorentz force law. Re-parameterising by the beam-line coordinate z and defining slopes $t_x \equiv dx/dz$, $t_y \equiv dy/dz$, the arc-length element is

$$N \equiv \sqrt{1 + t_x^2 + t_y^2}, \tag{1}$$

and the state vector $\mathbf{y} = (x, y, t_x, t_y, q/p)^\top$ (with $q/p \equiv q/p$) obeys the coupled ODEs

$$\frac{dx}{dz} = t_x, \tag{2}$$

$$\frac{dy}{dz} = t_y, \tag{3}$$

$$\frac{dt_x}{dz} = \kappa\, N \left[\, t_y \left(B_z + t_x B_x\right) - \left(1 + t_x^2\right) B_y \,\right], \tag{4}$$

$$\frac{dt_y}{dz} = \kappa\, N \left[\, -t_x \left(B_z + t_y B_y\right) + \left(1 + t_y^2\right) B_x \,\right], \tag{5}$$

with $\kappa = q/p\, c_{\text{light}}$. Momentum conservation implies $d(q/p)/dz = 0$ in the absence of energy loss, which is the regime used for training (no material description). Equations (2)–(5) are integrated with RK4 at a fixed spatial step of 5 mm inside `rk4_propagator.py`, with field values obtained by trilinear interpolation from the tabulated map.

2.2 Field characterisation

The LHCb dipole field is represented by the `twodip.rtf` parametric two-dipole file, evaluated on a regular $81 \times 81 \times 146$ grid (Cartesian in x, y and z) and cached on disk as float32. A full analysis

of the field is documented in `experiments/gen_1/V1/data_generation/FIELD_ANALYSIS.md`; the essentials are:

- Peak field strength $|B_y| \approx 1.03\ T$ at $z \approx 5007\ mm$ (on-axis), falling to $\lesssim 0.01\ T$ at the entrance/exit of the fiducial z range;
- Transverse variation across the fiducial box ($|x| < 300\ mm$, $|y| < 250\ mm$) is at the few-percent level relative to the on-axis value;
- The $B_y(z)$ profile is approximately Gaussian around its maximum and is used in diagnostics, see Figure 1.

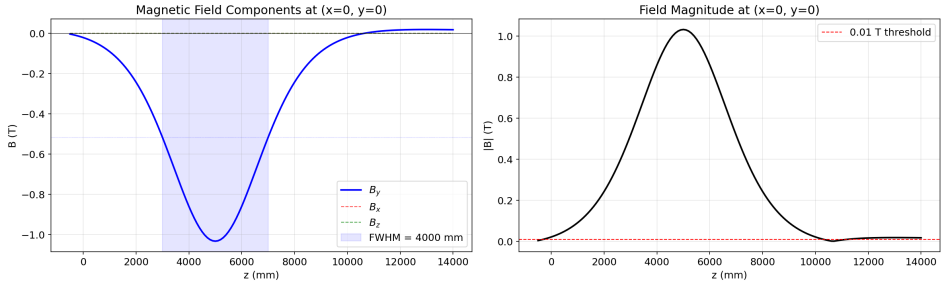

Figure 1: On-axis $B_y(z)$ profile of the LHCb dipole field extracted from the trilinearly interpolated `twodip.rtf` map ($81 \times 81 \times 146$ grid). The peak $|B_y| \approx 1.03\ T$ at $z \approx 5007\ mm$ defines the region of maximal trajectory bending and dominates the x-direction kicks that drive the asymmetry observed in the residuals (Section 7).

The fact that B_y strongly dominates over B_x and B_z in the fiducial region leads to a systematic prediction (verified in Section 7): the t_x slope receives a large kick from Eq. (4), while t_y is only coupled through small-B_x/B_z terms. The x-residuals propagate more than the y-residuals along Δz.

3 Data

3.1 Dataset layout

The training file `train_50M.npz` contains four arrays, all float32:

- $X \in \mathbb{R}^{50\,000\,000 \times 6}$ — initial state and step, columns $(x, y, t_x, t_y, q/p, \Delta z)$;
- $Y \in \mathbb{R}^{50\,000\,000 \times 4}$ — final state, columns $(x_f, y_f, t_{x,f}, t_{y,f})$;
- $P \in \mathbb{R}^{50\,000\,000}$ — scalar momentum (GeV/c);
- $Z \in \mathbb{R}^{50\,000\,000 \times 2}$ — $(z_{\text{start}}, z_{\text{end}})$ for traceability.

Input and output ranges (*as measured on the on-disk file*) are summarised in Table 1.

3.2 Reproducibility caveat: variable-Δz regeneration

The file `train_50M.npz` that is currently on disk was generated by the *variable*-Δz version of `generate_data.py`, whose header explicitly notes: "KEY IMPROVEMENT over original V1: Variable Δz... The old V1 used fixed $\Delta z = 8000\ mm$." The 17-MLP sweep whose results are reported below was, however, run on the earlier fixed-Δz incarnation of the same path, which has since been overwritten *in place*.

This is a reproducibility flag, not a result bug: the quoted accuracies in Section 6 describe the network behaviour on the fixed-Δz data, while the on-disk file now reflects a different sampling

Field	Column	Range	Units
x	$X[:,0]$	$[-300,\ 300]$	mm
y	$X[:,1]$	$[-250,\ 250]$	mm
t_x	$X[:,2]$	$[-0.30,\ 0.30]$	—
t_y	$X[:,3]$	$[-0.25,\ 0.25]$	—
q/p	$X[:,4]$	$[-10^{-3},\ 10^{-3}]$	c/MeV
Δz	$X[:,5]$	$[100,\ 10\,000]$	mm
x_f	$Y[:,0]$	$[-3278,\ 3281]$	mm
y_f	$Y[:,1]$	$[-2735,\ 2737]$	mm
$t_{x,f}$	$Y[:,2]$	$[-0.301,\ 0.301]$	—
$t_{y,f}$	$Y[:,3]$	$[-0.301,\ 0.301]$	—
p	P	$[1,\ 100]$	GeV/c, log-uniform
z_{start}	$Z[:,0]$	$[0.001,\ 13\,900]$	mm
z_{end}	$Z[:,1]$	$[103,\ 14\,000]$	mm

Table 1: Empirical ranges of the `train_50M.npz` fields as measured on disk (2026-04-21). Values transcribed verbatim from `docs/reports/gen1_verified_numbers.md`. The q/p distribution is split 50/50 between the two polarities.

strategy. Any re-run of the sweep on the current file will not reproduce the exact numbers quoted here. This issue is addressed in `gen_2` by pinning immutable data snapshots for every training run (see the companion report `gen2_old_data_results.tex`).

3.3 Splits and normalisation

Training, validation and test splits are 80%/10%/10% of the loaded subset (size `max_samples`; see Section 6), partitioned with a fixed seed 42 using NumPy `permutation`. Features and targets are each z-scored on the training split and the same means and standard deviations are applied to all three splits. Normalisation is essential: the four output components $(x_f, y_f, t_{x,f}, t_{y,f})$ span approximately 3–4 orders of magnitude in absolute value ($|x_f| \lesssim 10^3$ mm, $|t_{x,f}| \lesssim 0.3$), and raw-space MSE would be dominated by position residuals to the near-total exclusion of slope residuals.

4 MLP architecture and loss

4.1 Architecture

All 17 models share the same overall topology: a fully connected feedforward stack

$$[6] \ \to\ [h_1] \ \to\ \cdots \ \to\ [h_L] \ \to\ [4], \tag{6}$$

with SiLU (a.k.a. swish, $x \cdot \sigma(x)$) activations after every hidden layer [4], no dropout, and Xavier/Glorot uniform initialisation of all weights [5]. Biases are initialised to zero.

The widths $(h_1, \ldots, h_L)$ are varied across 17 configurations spanning 4868–1.06e6 parameters (Table 2): two-layer networks from `2x64` to `2x1024`, three-layer networks from `3x64` to `3x512`, tapered pyramids (`128_64`, `256_128`, `512_256_128`, `1024_512_256`, `512_512_256`, `256_256_128`), and four-layer networks `4x128`, `4x256`. The choice of SiLU follows standard practice for smooth regression targets; we have not ablated it.

4.2 Component-weighted MSE loss

Working in normalised output space, the loss for a batch of B samples is a simple unweighted mean-squared error

$$\mathcal{L}(\boldsymbol{\theta}) \;=\; \frac{1}{4B}\sum_{b=1}^{B}\sum_{k=1}^{4}\big(\hat{y}_k^{(b)}(\boldsymbol{\theta}) - y_k^{(b)}\big)^2, \tag{7}$$

where $\hat{\mathbf{y}}$ are the z-scored predictions and $\mathbf{y}$ are the z-scored targets. In raw units this is equivalent to an inverse-variance weighted MSE, which gives each output component the same *relative* importance in the gradient. All evaluation metrics, by contrast, are reported in raw physical units after de-normalisation.

Training logs the four individual component MSEs as well as the total loss to MLflow for post-hoc diagnostics.

5 Training protocol

Every sweep run uses the following fixed protocol (verified per-run in `v1_sweep_results.json`):

- Optimiser: AdamW [6] with $\beta_1 = 0.9$, $\beta_2 = 0.999$, learning rate 10^{-3}, weight decay 10^{-4} (base Adam algorithm as in [7]);
- Scheduler: cosine annealing with a 5-epoch linear warm-up;
- Gradient clipping at global-norm 1.0;
- Batch size 4096; up to 500 epochs;
- Early stopping on the validation loss with patience 30 and `min_delta` $= 10^{-7}$;
- Fixed `max_samples` = 5e6 loaded from `train_50M.npz` (only 10% of the 50e6-sample file);
- Experiment tracking: MLflow, experiment name `V1_MLP_sweep`.

Hardware. Runs were distributed across the Nikhef HTCondor cluster using a mix of NVIDIA V100 and L40S GPUs, with the smallest architectures (`2x64`, `2x128`, `3x64`, `3x128`, `128_64`, `512_256_128`, `3x512`) dispatched to CPU workers; the `device` field of each JSON entry records which backend was used. CPU and GPU runs follow identical numerical protocols; we have not observed device-dependent accuracy differences at the precision reported in Table 2.

6 17-MLP sweep

6.1 Master results table

Table 2 collects all 17 runs in `experiments/gen_1/V1/analysis/v1_sweep_results.json`, sorted by parameter count. The columns `pos_mean`, `pos_95` and `pos_99` refer to the mean and 95/99-th percentile of the 3D position residual

$$\rho \;\equiv\; \sqrt{(x_f - \hat{x}_f)^2 + (y_f - \hat{y}_f)^2} \tag{8}$$

on the held-out test split (the z-component is defined by the request and has zero residual by construction). The slope error σ_{slope} is computed analogously on (t_x, t_y) jointly. The rightmost column flags whether the network fits within the 64 kB CUDA constant memory (16384 float32 parameters).

Architecture	Hidden	n_{params}	max_samples	$\overline{\rho}$ [mm]	ρ_{95} [mm]	ρ_{99} [mm]	$\overline{\sigma_s}$ [rad]	$\sigma_{s,95}$ [rad]	64 kB?
mlp_2x64	64^2	4 868	5.0M	1.302	2.630	3.573	2.58e-3	5.72e-3	yes
mlp_3x64	64^3	9 028	5.0M	1.017	2.969	4.303	7.06e-3	1.62e-2	yes
mlp_128_64	128, 64	9 412	5.0M	1.455	2.897	3.866	3.96e-3	8.58e-3	yes
mlp_2x128	128^2	17 924	5.0M	1.793	3.417	4.462	2.38e-3	4.97e-3	no
mlp_3x128	128^3	34 436	5.0M	1.349	3.242	4.944	3.81e-3	8.76e-3	no
mlp_256_128	256, 128	35 204	5.0M	1.263	2.896	4.181	3.60e-3	7.39e-3	no
mlp_4x128	128^4	50 948	5.0M	2.400	5.728	8.478	8.13e-3	1.82e-2	no
mlp_2x256	256^2	68 612	5.0M	1.601	2.954	3.509	2.53e-3	5.92e-3	no
mlp_256_256_128	256, 256, 128	100 996	5.0M	2.083	4.413	6.148	6.69e-3	1.41e-2	no
mlp_3x256	256^3	134 404	5.0M	2.071	5.568	7.760	3.99e-3	8.52e-3	no
mlp_512_256_128	512, 256, 128	168 324	5.0M	1.499	3.599	5.611	5.23e-3	1.21e-2	no
mlp_4x256	256^4	200 196	5.0M	2.121	4.232	5.386	5.28e-3	1.15e-2	no
mlp_2x512	512^2	268 292	5.0M	2.697	5.887	9.273	2.70e-3	5.66e-3	no
mlp_512_512_256	512, 512, 256	398 596	5.0M	1.934	3.987	5.220	4.66e-3	1.07e-2	no
mlp_3x512	512^3	530 948	5.0M	3.024	6.569	9.603	4.26e-3	9.86e-3	no
mlp_1024_512_256	1024, 512, 256	664 324	5.0M	2.796	6.101	9.522	3.79e-3	8.27e-3	no
mlp_2x1024	1024^2	1 060 868	5.0M	2.562	5.055	6.727	3.24e-3	6.73e-3	no

Table 2: V1 sweep results, sorted by parameter count. All runs share identical training hyperparameters; max_samples= 5e6 is fixed across the board (i.e. only 10% of the 50e6-sample dataset is used in every run). Position residual ρ is defined in Eq. (8); slope error σ_s is the joint (t_x, t_y) RMS. Source: v1_sweep_results.json.

6.2 Reading the table

A few features are worth noting before the deeper analysis in Section 7:

- The best mean position error in the sweep is $\overline{\rho} = 1.02\ mm$ at mlp_3x64 (9028 params); the best 95-percentile is $\rho_{95} = 2.63\ mm$ at mlp_2x64 (4868 params). Both are 64 kB-deployable.
- Increasing parameter count does *not* monotonically improve accuracy. Architectures above 10^5 parameters are systematically *worse* on $\overline{\rho}$ than some of the smallest networks — for example, mlp_3x512 (530948 params) reaches only $\overline{\rho} = 3.02\ mm$. This is the fixed-budget confound: the larger networks under-fit the 5M-sample slice within 500 epochs.
- Four-layer networks (4x128, 4x256) pay a generalisation penalty for the extra depth without the sample budget to exploit it, and sit well above the Pareto front (Section 7).
- Slope errors bottom out around $\overline{\sigma_s} \approx 2.4{\times}10^{-3}$ rad (mlp_2x128); the dedicated two-hidden-layer widths 2x256 and 2x512 reach similar slope accuracy, suggesting the slope channel saturates well before the position channel.

7 Analysis

7.1 Pareto frontier

Figure 2 shows the accuracy-capacity frontier with the 64 kB boundary overlaid. The frontier is broken: small deployable networks sit slightly above the front defined by the mid-sized non-deployable ones, and the very large networks are strictly dominated — a signature of the fixed-budget confound discussed above, and a motivation for the gen_2 larger-Δz dataset that trains on the full 50e6 samples.

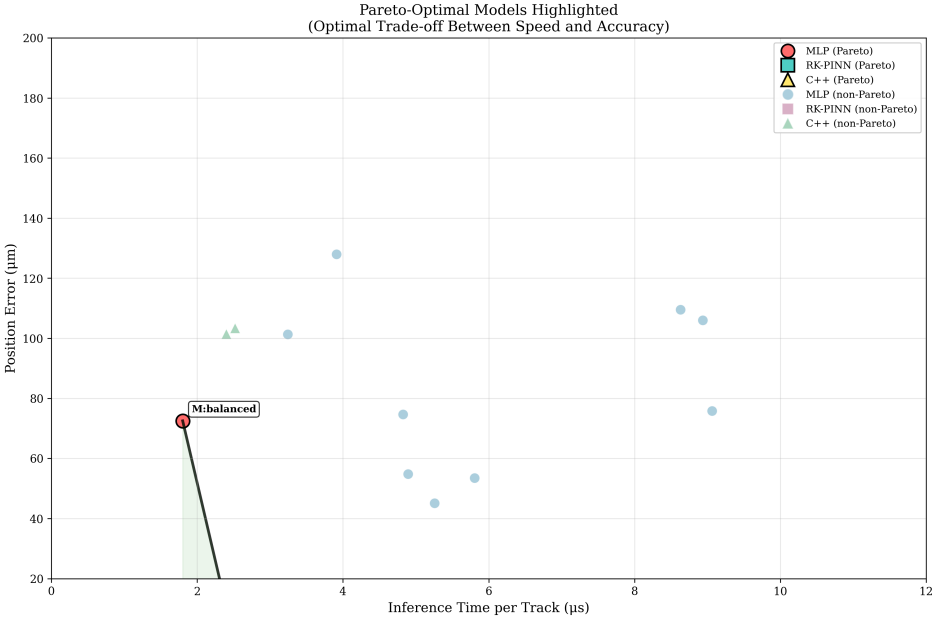

Figure 2: Pareto frontier of $\overline{\rho}$ versus parameter count for the 17 sweep architectures. The vertical boundary at 16384 parameters marks the 64 kB CUDA constant-memory budget: architectures to the left are deployable in Allen HLT1 constant memory; architectures to the right would have to live in global memory and would pay the associated latency cost. Note the non-monotonic behaviour above 10^5 parameters, driven by under-training relative to the 10^6-parameter capacity at a fixed 5e6-sample budget.

7.2 Convergence

Figure 3 confirms the reading of Table 2: convergence is clean for the small architectures and evidently incomplete for the large ones within the 5M-sample budget.

7.3 Error versus physics

Figure 4 (left) shows the test-set position error as a function of particle momentum p, and Figure 4 (right) decomposes the residual into x-only and y-only components for a representative model.

The x/y asymmetry is not merely a curiosity: it is a direct physics signature of the LHCb dipole geometry and provides a sanity check on the training. A network that happened to produce symmetric x/y residuals would have learned a field that does not match the real detector.

7.4 Slope errors and Kalman implications

The slope accuracy $\overline{\sigma_s} \sim 4\times10^{-3}$ rad achieved across the competitive architectures translates, via t_x, t_y, into a momentum-resolution-equivalent $\Delta p/p$ that is several times larger than the existing RK4 extrapolator on the same inputs. For ingestion by the downstream Kalman filter, which weights each hit by the extrapolated covariance, a slope error at this level would inflate the track-fit χ^2 and degrade the ghost rejection downstream. Closing this gap without blowing the parameter budget is the central goal of the `gen_2` work.

8 Throughput and deployment

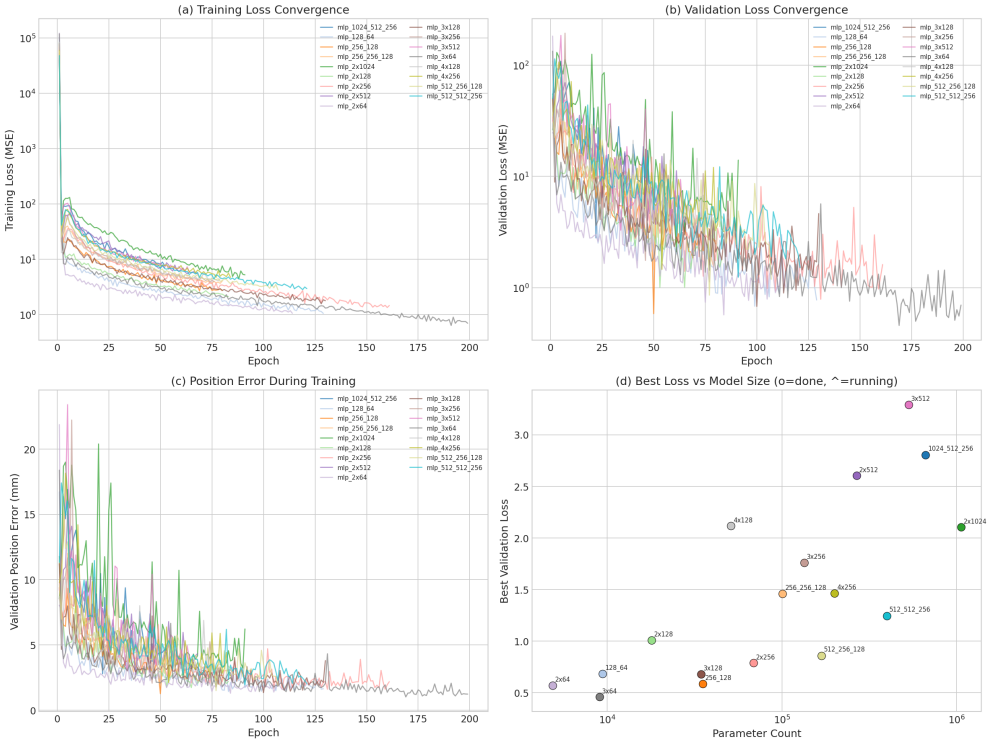

Figure 3: Training and validation loss trajectories for the 17 sweep runs. Small networks (bottom rows) converge within 50–100 epochs and then flatten under early stopping; the largest networks (top rows) are still improving when the 500-epoch cap triggers, consistent with the fixed-budget under-training hypothesis.

8.1 Benchmarks

Figures 5 and 6 report the inference throughput measurements for the sweep architectures (test-time only, excluding data transfer). The smallest constant-memory-deployable networks reach sub-microsecond per-track latencies on V100 and L40S, placing them within striking distance of the existing RK4 kernel that they would replace.

8.2 C++ integration path

The deployment path from PyTorch to Allen is:

1. `export_to_c_binary.py` exports the trained weights (and the z-score normalisation constants) as a flat little-endian binary blob next to the checkpoint;
2. `TrackMLPExtrapolator.cpp` (571 lines, Eigen-based) consumes that blob, implements the forward pass in dense matrix form, and exposes a C ABI suitable for wrapping into the Allen algorithm registry;
3. an identical reference implementation in pure NumPy is kept alongside for cross-validation against the PyTorch evaluation loop.

No deployment is performed in this report — the C++ path has been validated on CPU only, and the final integration into the Allen GPU pipeline is deferred to a later phase conditional on an MLP meeting both the accuracy and the 64 kB constraint simultaneously.

9 Limitations and transition

Four limitations of the `gen_1` sweep explicitly motivate the `gen_2` work:

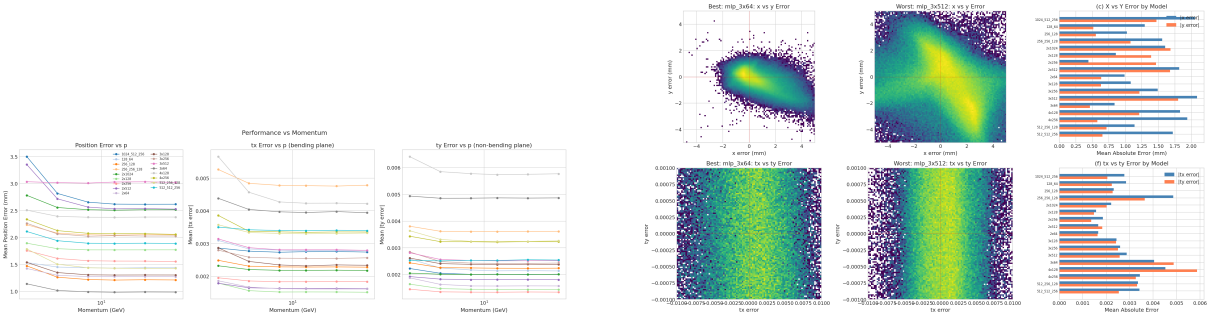

Figure 4: Left: mean ρ as a function of p. Low-momentum tracks ($p \lesssim 5\ GeV/c$) incur larger residuals because the q/p-scaled bending in Eqs. (4)–(5) is correspondingly larger, and small relative mis-estimates of the effective kick translate into proportionally larger absolute residuals. Right: x-only vs y-only residual distributions. The x component is systematically broader, consistent with the observation in Section 2 that the dominant field component B_y kicks t_x through Eq. (4), while t_y is only weakly coupled via the subdominant B_x, B_z terms; after integration over Δz, the x-residual therefore accumulates more than the y-residual.

(a) **Fixed-budget under-training.** Every sweep run uses `max_samples`$= 5e6$, i.e. 10% of the on-disk dataset. The largest networks clearly have not converged at this budget (Figure 3), so the measured accuracy/capacity scaling is biased in favour of the small networks and against the large ones. This must be corrected before drawing any general conclusion about MLP scaling for this task.

(b) **Best accuracy is not deployable.** The mid-to-large architectures that sit on or near the Pareto front all violate the 64 kB budget. The deployable subset (`2x64`, `3x64`, `128_64`) achieves $\overline{\rho} \gtrsim 1\ mm$, which is at the edge of what the Kalman filter tolerates at the VELO exit.

(c) **No physics regularisation.** The network is trained purely by output-space MSE and has no gradient-level awareness of Eqs. (2)–(5). In particular, there is no enforcement of momentum conservation, no penalty against predictions that violate the energy surface $\|\hat{t}\|^2 \leq \|t\|^2_{\max}$, and no RK4-consistency term.

(d) **Slope accuracy too loose for Kalman.** As discussed in Section 7, the plateau $\overline{\sigma_s} \sim 4\times10^{-3}$ rad is insufficient for tight downstream track fitting.

The follow-on companion work is split in two:

- `gen2_old_data_results.tex` retrains on the same dataset with physics-informed baselines — a PINN with an ODE residual loss of the form in Eqs. (2)–(5) following [8, 9, 10], and an RK_PINN that explicitly embeds the four-stage Butcher tableau into the forward pass. This targets limitations (c) and (d).
- `gen2_new_data_results.tex` extends the training set to cover the LHCb VELO inter-sensor spacing regime $\Delta z \in [25, 30]$ mm and extends momentum up to $p = 200\ GeV/c$, introduces the v2 fixes (PINN_v2, NeuralRK4) motivated by neural-ODE style residuals [11], and trains on the full dataset to eliminate the fixed-budget confound. This targets limitations (a) and (b).

10 Reproducibility

All results in this report are reproduced from:

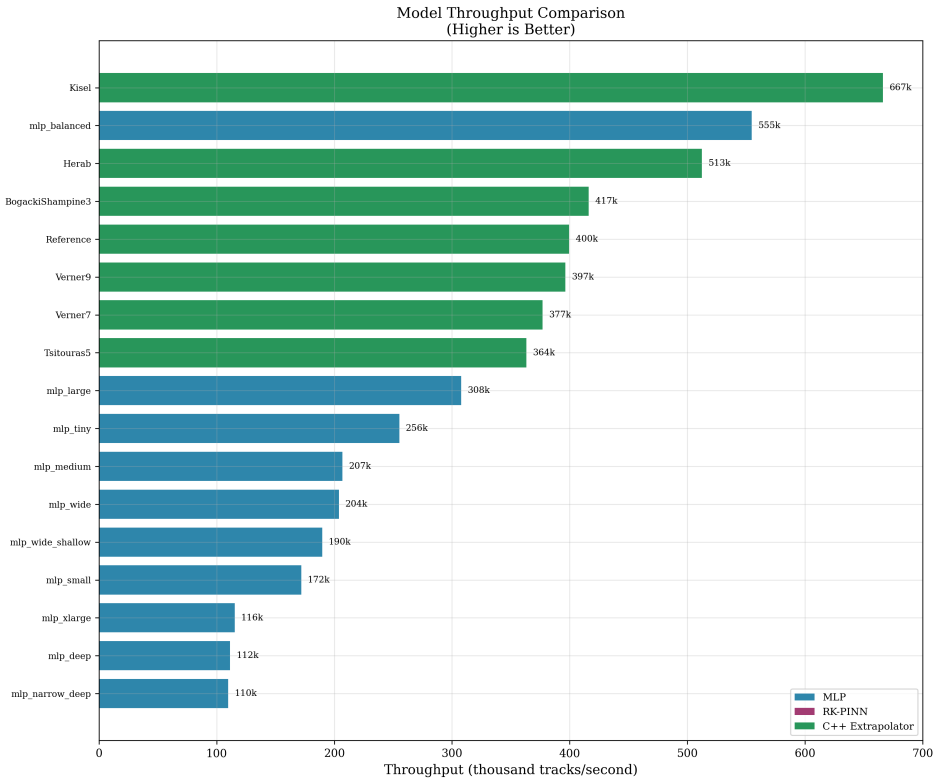

Figure 5: Inference throughput comparison across the 17 sweep architectures. The small deployable networks (left) achieve throughputs within an order of magnitude of the RK4 baseline.

- Environment: `conda activate TE` (Python 3.10, PyTorch 2.9.1+cu128); see `TrackExtrapolation/enviro` for the full pinning;
- Random seed: 42 (applied to NumPy, PyTorch, and the CUDA RNG);
- MLflow experiment: `V1_MLP_sweep` under the default local tracking URI;
- Sweep JSON: `experiments/gen_1/V1/analysis/v1_sweep_results.json`;
- Data generation: `experiments/gen_1/V1/data_generation/{generate_data.py, rk4_propagator.py}`
- Analysis notebook: `experiments/gen_1/V1/analysis/v1_model_characterisation.ipynb`;
- C++ reference: `TrackExtrapolation/src/TrackMLPExtrapolator.cpp` (Eigen, 571 lines); export pipeline `experiments/gen_1/V1/analysis/export_to_c_binary.py`.

Per the caveat in Section 3, exact re-run of the sweep on the *current* `train_50M.npz` will not reproduce the numbers in Table 2 at the quoted precision; to reproduce, the fixed-Δz data generator must be restored from git history first.

References

[1] LHCb Collaboration, "The LHCb Detector at the LHC," *JINST* **3**, S08005 (2008). DOI: 10.1088/1748-0221/3/08/S08005.

[2] R. Aaij *et al.*, "Allen: A high-level trigger on GPUs for LHCb," *Comput. Softw. Big Sci.* **4**, 7 (2020). doi:10.1007/s41781-020-00039-7.

[3] R. Frühwirth, "Application of Kalman filtering to track and vertex fitting," *Nucl. Instrum. Meth. A* **262**, 444–450 (1987).

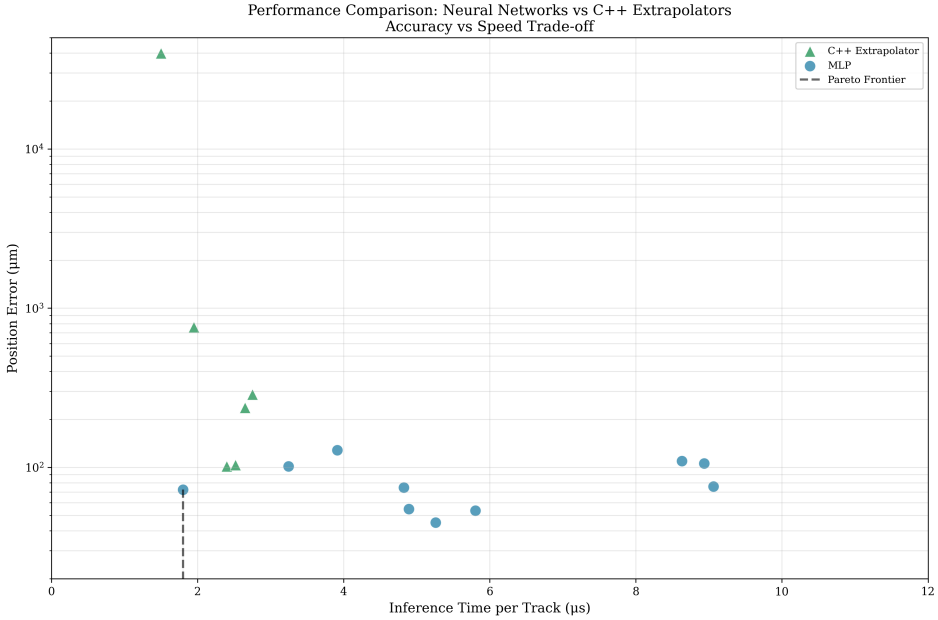

Figure 6: Accuracy vs wall-time for the full sweep. The 64 kB-deployable region is the lower-left corner. The set of Pareto-optimal deployable points defines the achievable accuracy/latency envelope for a purely data-driven MLP in the Allen HLT1 chain under the current sampling strategy.

[4] S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Networks* **107**, 3–11 (2018). arXiv:1702.03118.

[5] X. Glorot, Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," *AISTATS* (2010).

[6] I. Loshchilov, F. Hutter, "Decoupled weight decay regularization," *ICLR* (2019). arXiv:1711.05101.

[7] D. P. Kingma, J. Ba, "Adam: A method for stochastic optimization," *ICLR* (2015). arXiv:1412.6980.

[8] M. Raissi, P. Perdikaris, G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.* **378**, 686–707 (2019). doi:10.1016/j.jcp.2018.10.045.

[9] S. Wang, Y. Teng, P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM J. Sci. Comput.* **43**, A3055–A3081 (2021). doi:10.1137/20M1318043.

[10] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, M. W. Mahoney, "Characterizing possible failure modes in physics-informed neural networks," *NeurIPS* (2021). arXiv:2109.01050.

[11] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, "Neural ordinary differential equations," *NeurIPS* (2018). arXiv:1806.07366.